

RW BLE iOS SDK使用说明文档

1. 简介

本文档主要对SDK里所提供的功能接口与使用场景进行解释说明。

此文档仅适用于RW公司的蓝牙设备。

1.1 适用平台与语言

- iOS12及以上, 语言Objective-C.

1.2 相关术语

- App: 本文指的是手机端或平板电脑上运行的应用程序;
- 设备: 本文指的是可穿戴式硬件设备:如手表,戒指等;
- 上传: 指的是设备向App发送数据;
- 下发: 指的是App向设备发送数据;

1.3 注意事项

1. 此文档只用Objective-C语言进行说明;如果是使用swift语言开发,需要在项目的 `Bridging Header` 文件中导入相应的Objective-C 头文件;
2. SDK不提供模拟器版本, 因为在模拟器上无法调试BLE且我们依赖的第三方库部分也不支持模拟器环境下运行。

2. 快速开始 (Quick Start)

第1步: 获取最新版本的Xcode

要想使用 RW BLE SDK for iOS 开发项目, 您需要安装Xcode.

第2步: 手动部署添加依赖库

手动添加 `DHBleSDK.framework` 到工程;

Frameworks, Libraries, and Embedded Content

Name	Embed
 DHBleSDK.framework	Do Not Embed
 DHFoundation.framework	Embed & Sign
 DHUIKit.framework	Embed & Sign

第3步: 配置 info.plist文件

```
//需要在info.plist中对使用蓝牙进行说明
NSBluetoothAlwaysUsageDescription
NSBluetoothPeripheralUsageDescription
```

第4步: 初始化SDK

```
//AppDelegate里初始化DHBLESDK库
- (void)initBLESDK{
    [DHBLECentralManager setLogStatus:YES];
    [DHBLECentralManager initWithServiceUuids:@[]];

    //Demo里工具类初始化,可选择;
    [DHBluetoothManager sharedInstance];
}
```

⚠ Caution

日志开启后 `[DHBLECentralManager setLogStatus:YES]`; 日志文件将保存在 Document/DeviceLog 文件夹下。

3. 接口说明 (API Reference)

3.1 设备搜索与连接, 绑定与重连

3.1.1 搜索蓝牙

接口说明: 搜索蓝牙设备需使用 `[DHBLECentralManager startScan]`; 并实现 `DHBLEConnectDelegate` 回调接口;

返回的 `DHPeripheralModel` 实体如果 `model.macAddr` 为空, 代表手机设置是已配对设备

```
// 1. 开始搜索
[DHBLECentralManager startScan];

//2. 设备委托
[DHBLECentralManager sharedInstance].connectDelegate = self;

//3. DHBLEConnectDelegate接口会回调搜索到的蓝牙设备
- (void)centralManagerDidDiscoverPeripheral:(NSArray <DHPeripheralModel
*>*)peripherals
```

3.1.2 停止搜索

接口说明: 停止搜索蓝牙设备

```
[DHBLECentralManager stopScan];
```

3.1.3 连接设备

接口说明: 连接指定设备, 并设置设备连接回调 `DHBLEConnectDelegate`.

```
// 1. 初始化并注册回调
[DHBLECentralManager sharedInstance].connectDelegate = self;

// 2. 连接设备
DHPeripheralModel *deviceModel = self.deviceArray[indexPath.row];
[DHBLECentralManager connectDeviceWithModel:deviceModel];

// 3. 实现并回调蓝牙连接状态
@protocol DHBLEConnectDelegate <NSObject>
```

`DHBLEConnectDelegate` 接口说明:

方法	说明
<code>centralManagerDidDiscoverPeripheral</code>	<code>startScan</code> 调用后,搜索到设备后会回调返回
<code>centralManagerDidConnectPeripheral</code>	<code>connectDeviceWithModel</code> 后,连接成功会返回.
<code>centralManagerDidFunctionMenu</code>	成功获取设备配置表后会返回;业务操作应该在此之后操作.
<code>centralManagerDidDisconnectPeripheral</code>	蓝牙断开会回调
<code>centralManagerDidFailedPeripheral</code>	蓝牙失败会回调
<code>centralManagerDidUpdateState</code>	蓝牙开关状态变化会回调

Tip

连接后, 业务操作应该在 `centralManagerDidFunctionMenu` 之后才进行操作.

(1) 监听 `BluetoothNotificationConnectStateChange` 也可获取连接状态变化

```
[[NSNotificationCenter defaultCenter] addObserver:self
selector:@selector(connectStateChange:)
name:BluetoothNotificationConnectStateChange object:nil];

- (void)connectStateChange:(NSNotification *)ntf
{
    NSLog(@"NewHomeController connectStateChange");
    if ([DHBluetoothManager sharedInstance].isConnected){
        self.infoDeviceStateLb.text = @"Connected";
    }
    else{
        self.infoDeviceStateLb.text = @"Disconnected";
    }
}
```

(2) `[DHBLECentralManager isConnected]` 可以获取是否连接成功.

3.1.4 断开连接设备

接口说明: 断开正在连接的设备;

```
[DHBLECentralManager disconnectDevice];
```

3.1.5 绑定与自动重连,解绑

3.1.5.1 DHBleCentralManager 本地保存当前设备

设置后将保存当前设备的UDID,将会自动重新连接.

方法说明:

```
+(void)setBindedStatus:(BOOL)isBinded
```

调用示例:

```
//保存本地, 重新打开将重连  
[DHBleCentralManager setBindedStatus:YES];
```

3.1.5.2 DHBleCentralManager 删除本地保存

设置后将删除当前设备的UDID, 类似于解除本地绑定.

方法说明:

```
+(void)setBindedStatus:(BOOL)isBinded
```

调用示例:

```
[DHBleCentralManager setBindedStatus:NO];  
[DHBleCentralManager disconnectDevice];
```

3.1.6 设备配置表

Important

因设备型号多, 支持的功能不同,所以引入功能表信息,可查询设备功能支持情况. 具体参考 DeviceFuncV2Model类. 可根据业务自行保存功能表内容.

```
-(void)centralManagerDidFunctionMenu:(DeviceFuncV2Model *)deviceFuncModel
```

DeviceFuncV2Model类属性定义:

属性	说明
isPushMsgEnableSwitch	是否启用消息控制开关
isAlarm	是否支持闹钟
isBackLight	是否支持屏幕睡眠时间设置
isSupportWorkout3	是否支持多运动;
isSupportMuslimCountSwitch	是否支持Muslim赞念开关
isSupportHrSpO2Alert	是否支持HR,SP02报警提示功能
isSupportMotoVibrationLevel	是否支持马达震动提醒
isDataTypeActivity	是否支持计步
isDataTypeHeart	是否支持心率
isDataTypeBloodPressure	是否支持血压
isDataTypeSleep	是否支持睡眠
isDataTypeSPO2	是否支持血氧
isDataTypeHRV	是否支持心率变异性
isDataTypeStress	是否支持压力
isDataTypeBloodSugar	是否支持血糖
isDataTypeMuslimCount	是否支持赞念

3.2 设备功能操作

调用接口发送错误码定义 SendStateCode:

SendStateCode	值	说明
SendState_OK	0	成功
SendState_BLENoOpen	1	失败, 蓝牙未打开
SendState_BLEDisconnect	2	失败, 蓝牙未连接
SendState_TimeOut	3	失败, 超时
SendState_Failed	4	失败
SendState_DuplicateSend	5	有重复指令
SendState_NotSupport	6	设备不支持

3.2.1 基础功能指令接口

3.2.1.0 Get SDK Version

获取SDK版本号.

方法说明:

[DHBLECommand getSDKVersion]

调用示例:

```
NSLog(@"%@", [DHBLECommand getSDKVersion]);
```

3.2.1.1 获取蓝牙Mac地址

因iOS系统配对后,广播包无法拿到mac地址,提供此方法可获取.

方法说明:

+ (void)ringGetMacAddress:(void(^)(int code, id data))block

返回参数说明:

参数	类型	说明	值
model	DHDeviceInfoModel	类	macAddr: Mac地址

调用示例:

```
[DHBLECommand ringGetMacAddress:^(int code, id _Nonnull data) {
    DHDeviceInfoModel *tDeviceInfoData = data;
    NSLog(@"mac: %@", tDeviceInfoData.macAddr);
}];
```

3.2.1.2 设置用户信息

用户信息设置与计步卡路里,距离有关; 设备初始化时性别为1, 年龄 18, 身高170cm, 体重 65 kg.

方法说明:

+ (void)setUserInfo:(DHUserInfoSetModel *)model block:(void(^)(int code, id data))block

参数说明:

参数	类型	说明	值
model	DHUserInfoSetModel	类	gender: 性别 (0.女 1.男) height: 身高cm,浮点型 weight: 体重kg,浮点型 stepGoal: 计步目标值,暂无功能 age: 年龄

调用示例:

```
DHUserInfoSetModel *userInfoModel = [[DHUserInfoSetModel alloc] init];
userInfoModel.gender = 1;
userInfoModel.height = 170;
userInfoModel.weight = 600;
userInfoModel.stepGoal = 8000;
userInfoModel.age = 20;
[DHBLECommand setUserInfo:userInfoModel block:^(int code, id _Nonnull data) {
    if (code == 0){
        NSLog(@"set ok");
    }
}];
```

3.2.1.3 获取固件信息

接口说明: 获取固件型号,固件版本号,UI版本号;

```
[DHBLECommand getFirmwareVersion:^(int code, id _Nonnull data) {
    if (code == 0){
        NSLog(@"getFirmwareVersion OK");
        DHFirmwareVersionModel *model = data;
        NSLog(@"型号 %@ 固件版本 %@ UI版本 %@", model.deviceModel,
model.firmwareVersion, model.uiVersion);
    }
}];
```

3.2.1.4 获取电量

接口说明: app获取设备电量

```
[DHBLECommand getBattery:^(int code, id _Nonnull data) {
    if (code == 0){
        DHBatteryInfoModel *model = data;
        NSLog(@"getBattery OK 电量值 %zd", model.battery);
    }
}];
```

3.2.1.5 获取与设置视频控制开关

设置是否开启戒指手势控制刷视频; 此功能需配对蓝牙HID.

```

//设置视频控制开关
DHVideoHidSetModel *tModeSetModel = [[DHVideoHidSetModel alloc] init];
tModeSetModel.isOpen = YES;
[DHBLECommand setVideoHid:tModeSetModel block:^(int code, id _Nonnull data) {
    if (code == 0){
        NSLog(@"setVideoHid OK");
    }
}];

// 获取视频控制开关
[DHBLECommand getVideoHid:^(int code, id _Nonnull data) {
    if (code == 0){
        DHVideoHidSetModel *model = data;
        NSLog(@"getVideoHid OK 开关 %d", model.isOpen);
    }
}];

```

3.2.1.6 获取与设置LED亮屏强度

方法说明:

```
+(void)getRingLEDLight:(void(^)(int code, id data))block;
```

```
+(void)setRingLEDLight:(DHLedLightSetModel *)model block:(void(^)(int code, id data))block;
```

参数说明:

参数	类型	说明	值
model	DHLedLightSetModel	类	isOpen: false为off,ture为(1-3Level) lightLevel: 1微光, 2柔光, 3强光

调用示例:

```

// 获取LED亮屏强度
[DHBLECommand getRingLEDLight:^(int code, id _Nonnull data) {
    if (code == 0){
        DHLedLightSetModel *model = data;
        NSLog(@"getRingLEDLight OK 开关 %d", model.isOpen);
    }
}];

//设置LED亮屏强度
DHLedLightSetModel *tModeSetModel = [[DHLedLightSetModel alloc] init];
tModeSetModel.isOpen = YES; //设置为NO关闭
tModeSetModel.lightLevel = 3; //1微光2柔光3强光
[DHBLECommand setRingLEDLight:tModeSetModel block:^(int code, id _Nonnull data)
{
    if (code == 0){
        NSLog(@"setRingLEDLight OK");
    }
}];

```


3.2.1.7 获取与设置佩戴位置

```
// 获取佩戴位置
[DHBluetoothCommand getRingWearHand:^(int code, id _Nonnull data) {
    if (code == 0){
        NSInteger tWearHand = [data intValue]; //0左手 1右手
        NSLog(@"getRingWearHand OK 佩戴位置 %zd", tWearHand);
    }
}];

//设置佩戴位置
uint8_t tModeSetModel = 0; //0左手 1右手
[DHBluetoothCommand setRingWearHand:tModeSetModel block:^(int code, id _Nonnull data)
{
    if (code == 0){
        NSLog(@"setRingWearHand OK");
    }
}];
```

3.2.1.8 启动与关闭拍照

启动拍照功能后,设备可通过手势控制app自定义相机拍照.

配置表属性: `isTakePhoto`

`BluetoothNotificationCameraTakePicture` 设备发出拍照通知,进行拍照

```
//APP进相机界面启动 1为控制设备进对应界面, 0为控制设备退出
[DHBluetoothCommand controlCamera:1 block:^(int code, id _Nonnull data) {

}];

//监听设备发出拍照指令
[[NSNotificationCenter defaultCenter] addObserver:self
selector:@selector(cameraTakePictureNotification)
name:BluetoothNotificationCameraTakePicture object:nil];

- (void)cameraTakePictureNotification {
    //进行拍照
    NSLog(@"cameraTakePictureNotification 进行拍照");
}
```

3.2.1.9 查找设备

调用查找后设备灯或屏幕会亮.

方法说明:

```
+(void)controlFindDeviceBegin:(void(^)(int code, id data))block;
```

调用示例:

```
[DHBleCommand controlFindDeviceBegin:^(int code, id _Nonnull data) {

}];
```

3.2.1.10 关机,恢复出厂设置

方法说明:

```
+(void)controlDevice:(NSInteger)type block:(void(^)(int code, id data))block;
```

参数说明:

参数	类型	说明	值
type	NSInteger	整形	1: 关机 2: 恢复出厂

调用示例:

```
// 1关机 2 恢复出厂
[DHBleCommand controlDevice:1 block:^(int code, id _Nonnull data) {

}];
```

3.2.1.11 闹钟

配置表属性: isAlarm

3.2.1.11.1 获取已设置闹钟

方法说明:

```
+(void)getAlarms:(void(^)(int code, id data))block
```

调用示例:

```
//获取设备里已保存的闹钟
[DHBleCommand getAlarms:^(int code, id _Nonnull data) {
    if (code == 0) {
        NSArray *tAlarmList = data;
        NSLog(@"getAlarms %zd", tAlarmList.count);
    }
}];
```

3.2.1.11.2 设置闹钟

当前协议不支持单独修改闹钟，任何单个闹钟的开关或删除操作，均需重新下发全部闹钟配置。

方法说明:

```
+(void)setAlarms:(NSArray <DHAlarmSetModel *>*)alarms block:(void(^)(int code, id data))block;
```

参数说明:

参数	类型	说明	值
alarms	List	闹钟数组	isOpen: true开/false关 repeats: IntArray(7)周日至周六,要重复的对应置1 hour: 闹钟开始时 minute: 闹钟开始分

调用示例:

```
//设置闹钟
DHAlarmSetModel *tAlarm1 = [[DHAlarmSetModel alloc] init];
tAlarm1.hour = 07; //时
tAlarm1.minute = 00; //分
tAlarm1.isOpen = true; //开关
tAlarm1.repeats = @[0,0,0,0,0,0,1]; //重复周期,周六重复

DHAlarmSetModel *tAlarm2 = [[DHAlarmSetModel alloc] init];
tAlarm2.hour = 8;
tAlarm2.minute = 00;
tAlarm2.isOpen = true;
tAlarm2.repeats = @[]; //单次闹钟

[DHBleCommand setAlarms:@[tAlarm1, tAlarm2] block:^(int code, id _Nonnull data)
{

}];
```

3.2.1.11.3 删除所有闹钟

setAlarms 参数传空数组,即可删除所有闹钟

```
[DHBleCommand setAlarms:@[] block:^(int code, id _Nonnull data) {

}];
```

3.2.1.12 震动次数设置与获取

设置设备震动次数;
配置表属性: isSupportMotoVibrationLevel

方法说明:

```
+(void)setRingMotorLevel:(NSInteger)motorLevel motorNum:(NSInteger)motorNum block:
(void(^)(int code, id data))block;

+(void)getRingMotorLevel:(void(^)(int code, id data))block;
```

参数说明:

参数	类型	说明	值
motorLevel	Int	整形	震动强度, 0:关闭 1:低 2: 中 3: 高; 未定义此功能的可忽略
motorNum	Int	整形	震动次数可以被设置(0-6次), 初始默认2次.设置0次不震动

调用示例:

```
//设置
[DHBluetooth setRingMotorLevel:1 motorNum:2 block:^(int code, id _Nonnull data)
{
    if (code == 0){
        NSLog(@"set ok");
    }
}]];

//获取
[DHBluetooth getRingMotorLevel:^(int code, id _Nonnull data) {
    if (code == 0){
        DHVibrationLevelModel *tVibrationModel = data;
        NSLog(@"getRingMotorLevel Level %d num %d", tVibrationModel.vibrationLevel,
tVibrationModel.vibrationNumber);
    }
}]];

```

3.2.1.13 屏幕睡眠模式设置与获取

- 设置屏幕睡眠开启与时间;
- 配置表属性: isBackLightSleepMode

方法说明:

```
+(void)setDisplaySleepMode:(DHBrightTimeSetModel *)model block:(void(^)(int code, id data))block

+(void)getDisplaySleepMode:(void(^)(int code, id data))block

```

参数说明:

参数	类型	说明	值
DHBrightTimeSetModel	类		sleepOpen, 开关打开YES或关闭NO sleepStartHour, 开始时间小时 sleepStartMin,开始时间分钟 sleepEndHour,结束时间小时 sleepEndMin,结束时间分钟

调用示例:

```

//设置
DHBrightTimeSetModel *sleepModel = [[DHBrightTimeSetModel alloc] init];
sleepModel.sleepOpen = YES;
sleepModel.sleepStartHour = 20;
sleepModel.sleepStartMin = 00;
sleepModel.sleepEndHour = 06;
sleepModel.sleepEndMin = 00;
[DHBLECommand setDisplaySleepMode:sleepModel block:^(int code, id _Nonnull data)
{

}];

//获取
[DHBLECommand getDisplaySleepMode:^(int code, id _Nonnull data) {
    if (code == 0){
        DHBrightTimeSetModel *tModel = data;
        NSLog(@"getDisplaySleepMode sleepOpen %d sleepStartHour %d sleepStartMin %d",
tModel.sleepOpen, tModel.sleepStartHour, tModel.sleepEndMin);
    }
}];

```

3.2.1.14 消息推送开关设置与获取

消息推送开关;

配置表属性: `isPushMsg`, `pushMsgSwitchValue` 为确定设备哪些应用支持.

方法说明:

```
+(void)ringSetAncs:(DHAncsSetModel *)model block:(void(^)(int code, id data))block
```

```
+(void)ringGetAncs:(void(^)(int code, id data))block
```

参数说明:

参数	类型	说明	值
DHAncsSetModel	类		见DHAncsSetModel类定义

调用示例:

```

//设置
DHAncsSetModel *tAncsModel = [[DHAncsSetModel alloc] init];
tAncsModel.isSMS = YES;
[DHBLECommand ringSetAncs:tAncsModel block:^(int code, id _Nonnull data) {

}];

//获取
[DHBLECommand ringGetAncs:^(int code, id _Nonnull data) {
    if (code == 0){
        DHAncsSetModel *ancsModel = data;

```

```

    }
}];

//按下面的顺序确定当前设备支持哪些应用消息.
UInt8 tBitRow = i;
if (i == 24){ //其它
    tBitRow = 0;
}
if (i > 0 && (tMsgSwitchValue & (1 << tBitRow)) < 1){ //不支持的消息全设置为false,不管从设备读到的值.
    continue;
}

self.msgOpenFlagArr = [NSMutableArray arrayWithArray:@[
    @(self.model.isCall),
    @(self.model.isSMS),
    @(self.model.isEmail),
    @(self.model.isSkype),
    @(self.model.isFacebook),
    @(self.model.isWhatsapp),
    @(self.model.isLine),
    @(self.model.isInstagram),
    @(self.model.isKakaotalk),
    @(self.model.isGmail),
    @(self.model.isTwitter),
    @(self.model.isLinkedin),
    @(self.model.isJLSinaWeiBo),

    @(self.model.isQQ),
    @(self.model.isWechat),
    @(self.model.isJLBand),
    @(self.model.isJLTelegram),
    @(self.model.isJLBetween),
    @(self.model.isJLNavercafe),
    @(self.model.isYoutube),
    @(self.model.isJLNetflix), //(1<<21)
    @(self.model.isMax), // (1<<22)
    @(self.model.isVkim), // (1<<23)
    //          @(self.model.isMessenger),

    @(self.model.isOther)
]];

```

3.2.1.14 获取与设置赞念是否打开

设置赞念功能是否打开;

配置表属性: `isSupportMuslimCountSwitch`

方法说明:

```
+(void)setMuslimCountSwitch:(UInt8)isOpen block:(void(^)(int code, id data))block
```

```
+(void)getMuslimCountSwitch:(void(^)(int code, id data))block
```

参数说明:

参数	类型	说明	值
isOpen	Int		0: 关闭 1: 打开

调用示例:

```
//设置
[DHBLECommand setMuslimCountSwitch:1 block:^(int code, id _Nonnull data) {

}];

//获取
[DHBLECommand getMuslimCountSwitch:^(int code, id _Nonnull data) {
    if (code == 0){
        Boolean tOpen = [data boolValue];
        NSLog(@"getMuslimCountSwitch tOpen %d", tOpen);
    }
}];
```

3.2.1.15 获取与设置心率/血氧报警配置

设置心率与血氧通知报警数据功能; 报警提示会通 BluetoothNotificationRingHealthOverAlert 通知出来.

配置表属性: isSupportHrSpO2Alert

方法说明:

```
+(void)getHRAAlert:(void(^)(int code, id data))block

+(void)setHRAAlert:(DHHRAAlertModel *)overModel block:(void(^)(int code, id data))block

+(void)getSP02Alert:(void(^)(int code, id data))block

+(void)setSP02Alert:(DHHRAAlertModel *)overModel block:(void(^)(int code, id data))block
```

参数说明:

参数	类型	说明	值
DHHRAAlertModel	类		isOpen: YES 开, NO 关; overValue: 报警值, 默认值为 心率超过160, 血氧低于94%; underValue: 低于设置值报警; 如果获取到为0xff,代表不支持此项功能;

注意: 通过 `getHRAAlert()` 获取如果 `underValue` 为 `0xff`, 代表不支持此项功能.

调用示例:

```
//设置
DHHRAAlertModel *tHRAAlertModel = [[DHHRAAlertModel alloc] init];
tHRAAlertModel.isOpen = YES;
tHRAAlertModel.overValue = 160;
tHRAAlertModel.underValue = 0xff;
[DHBleCommand setHRAAlert:tHRAAlertModel block:^(int code, id _Nonnull data) {

}];

//获取
[DHBleCommand getHRAAlert:^(int code, id _Nonnull data) {
    if (code == 0){
        DHHRAAlertModel *model = data;
        NSLog(@"getHRAAlert %d %zd", model.isOpen, model.overValue);
    }
}];

//报警结果通知推送
[[NSNotificationCenter defaultCenter] addObserver:self
selector:@selector(healthOverAlert:)
name:BluetoothNotificationRingHealthOverAlert object:nil];

- (void)healthOverAlert:(NSNotification *)ntf
{
    NSDictionary *tUserInfo = ntf.userInfo;
    NSInteger tType = [tUserInfo[@"type"] integerValue];
    NSInteger tValue = [tUserInfo[@"value"] integerValue];

    if (tType == 0){ //HeartRate Alert
    }
    else if (tType == 1){ //SP02
    }
    else if (tType == 2){ //HeartRate Alert Under
    }
}
```

3.2.1.16 获取与设置亮屏时长

配置表属性: `isBackLight`

方法说明:

```
+(void)getBrightTime:(void(^)(int code, id data))block
```


`+(void)setBrightTime:(DHBrightTimeSetModel *)model block:(void(^)(int code, id data))block`

参数说明:

DHBrightTimeSetModel参数	类型	说明	值
duration	Int	亮屏时长,秒(s), 范围0-30s;	
durationNums	String	设备支持的时长值, 获取到如果有值,以逗号隔开;	

3.2.1.17 获取与设置抬腕亮屏时长

配置表属性: `isSupportRaisescreen`

方法说明:

`+(void)ringGetGesture:(void(^)(int code, id data))block;`

`+(void)ringSetGesture:(DHGestureSetModel *)model block:(void(^)(int code, id data))block;`

参数说明:

BrightScreenBean参数	类型	说明	值
isOpen	Int	true:开启; false: 关闭	
startHour	Int	开始时间小时	
startMin	Int	开始时间分钟	
endHour	Int	结束时间小时	
endMin	Int	结束时间分钟	

3.2.1.18 设置时间格式12/24小时制

带屏显示时间的设备才有效.

方法说明:

`+(void)ringSetTimeformat:(UInt8)timeformat block:(void(^)(int code, id data))block;`

参数说明:

参数	类型	说明	
timeformat	Int	0: 24小时制 1: 12小时制	

3.2.2 健康数据同步(实时单次与全天检测)

健康数据检测有两种方式: 实时单次检测与全天检测。健康数据包括心率,血氧,压力,HRV,睡眠等,睡眠无实时检测。

(1) 实时单次检测: APP侧启动设备进入单次检测,检测完后马上返回结果。

(2) 全天检测: 可设置间隔时间,比如30分钟或60分钟设备会进行检测并保存值; **app**一直不同步情况下,设备可保存**3-6天**的数据。

3.2.2.1 实时检测-启动与关闭设备健康数据检测

启动健康数据检测(心率,血氧,HRV,压力,血糖);

测试完成设备通过 `BluetoothNotificationHealthRingMeasureStateChange` 通知app;

测试实时值通过 `BluetoothNotificationHealthRingMeasureValueChange` 通知app;

方法说明:

```
+(void)controlOpen:(NSInteger)type dataType:(NSInteger)dataType block:(void(^)(int code, id data))block
```

参数说明:

参数	类型	说明	值
dataType	NSInteger	健康数据类型	心率: BLE_KEY_HEART_RATE 血氧: BLE_KEY_BLOOD_OXYGEN HRV: BLE_KEY_HRV 压力: BLE_KEY_STRESS 血糖: BLE_KEY_BLOOD_SUGAR
type	NSInteger	启动/关闭	启动: 1 关闭: 0

调用示例:

```
+ (void)controlOpen:(NSInteger)type dataType:(NSInteger)dataType block:(void(^)(int code, id data))block
//启动心率测试
[DHBLECommand controlOpen:1 dataType:BLE_KEY_HEART_RATE block:^(int code, id _Nonnull data) {

}];

//关闭心率测试
[DHBLECommand controlOpen:0 dataType:BLE_KEY_HEART_RATE block:^(int code, id _Nonnull data) {

}];

// 监听测量中实时数值改变
```

```

[[NSNotificationCenter defaultCenter] addObserver:self
selector:@selector(updateRingMeasureValueChange:)
name:BluetoothNotificationHealthRingMeasureValueChange object:nil];

- (void)updateRingMeasureValueChange:(NSNotification *)ntf
{
    NSDictionary *tUserInfo = ntf.userInfo;
    NSInteger tDataValue = [tUserInfo[@"dataValue"] integerValue];
    NSInteger tDataType = [tUserInfo[@"dataType"] integerValue];
    NSInteger tTimestamp = [tUserInfo[@"timestamp"] integerValue];

    NSLog(@"updateRingMeasureValueChange %04X 数值: %zd timestamp: %ld",
tDataType, tDataValue, tTimestamp);
    if (tDataType == BLE_KEY_APP_REAL_TIME_MUSLIM_COUNT){ //Muslim Count
    }
    else if (tDataType == BLE_KEY_APP_REAL_BLOOD_SUGAR_DATA){ //BloodSugar 血糖
    }
    else if (tDataType == BLE_KEY_APP_REAL_TIME_HRV_DATA){ //HRV 心率变导性
    }
    else if (tDataType == BLE_KEY_APP_REAL_TIME_HR_DATA){ //HR 心率
    }
    else if (tDataType == BLE_KEY_APP_REAL_TIME_BLOOD_OXYGEN_DATA){ //BloodOxygen
血氧
    }
    else if (tDataType == BLE_KEY_APP_REAL_TIME_STRESS_DATA){ //Stress 压力
    }
}
}

```

3.2.2.2 全天检测-设置健康数据全天监听间隔

设置健康数据(心率,血氧,HRV,压力,血糖)全天监听间隔, 单位分钟.

注意事项:暂间隔只有心率可设置30分钟与60分钟, 其它(血氧,HRV,压力,血糖)只能设置开与关; 开始与结束时间固定全天,不可修改.

3.2.2.2.1 心率检测设置与获取

间隔只有心率可设置30分钟与60分钟;

方法说明:

```

+(void)setHeartRateMode:(DHHeartRateModeSetModel *)model block:(void(^)(int code,
id data))block

```

```

+(void)getHeartRateMode:(void(^)(int code, id data))block

```

参数说明:

参数	类型	说明	值
model	DHHeartRateModeSetModel	类	isOpen: true开/false关 interval: 间隔时间30或60分钟 startHour: 0 固定0不能修改 startMin: 0 固定0不能修改 endHour: 23 固定23不能修改 endMin: 59 固定59不能修改;

调用示例:

```
// 1. Set HeartRate Monitor(设置心率监听)
DHHeartRateModeSetModel *tModeSetModel = [[DHHeartRateModeSetModel alloc] init];
tModeSetModel.isOpen = YES;
tModeSetModel.startHour = 00; //开始时间固定
tModeSetModel.startMinute = 00;
tModeSetModel.endHour = 23; //结束时间固定
tModeSetModel.endMinute = 59;
tModeSetModel.interval = 30; //可设置30分钟或60分钟
[DHBLECommand setHeartRateMode:tModeSetModel block:^(int code, id _Nonnull data)
{
    if (code == 0){
        NSLog(@"setHeartRateMode OK");
    }
}]];

//1. 获取心率监听
[DHBLECommand getHeartRateMode:^(int code, id _Nonnull data) {
    if (code == 0){
        DHHeartRateModeSetModel *model = data;
        NSLog(@"getHeartRateMode OK 开关 %d 检测周期 %d", model.isOpen,
model.interval);
    }
}]];

```

3.2.2.2.2 血氧检测设置与获取

间隔血氧只可设置60分钟;

方法说明:

```
+(void)setBoMode:(DHBoModeSetModel *)model block:(void(^)(int code, id data))block
+(void)getBoMode:(void(^)(int code, id data))block

```

参数说明:

参数	类型	说明	值
model	DHBoModeSetModel	类	isOpen: true开/false关 interval: 间隔时间固定60分钟 startHour: 0 固定0不能修改 startMin: 0 固定0不能修改 endHour: 23 固定23不能修改 endMin: 59 固定59不能修改;

调用示例:

```
// 2. Set Blood oxygen Monitor(设置血氧监听)
DHBoModeSetModel *tModeSetModel = [[DHBoModeSetModel alloc] init];
tModeSetModel.isOpen = YES;
tModeSetModel.startHour = 00; //开始时间固定
tModeSetModel.startMinute = 00;
tModeSetModel.endHour = 23; //结束时间固定
tModeSetModel.endMinute = 59;
tModeSetModel.interval = 60; //固定不可设置
[DHBLECommand setBoMode:tModeSetModel block:^(int code, id _Nonnull data) {
    if (code == 0){
        NSLog(@"setBoMode OK");
    }
}];

//2. 获取血氧监听
[DHBLECommand getBoMode:^(int code, id _Nonnull data) {
    if (code == 0){
        DHBoModeSetModel *model = data;
        NSLog(@"getBoMode OK 开关 %d 检测周期 %d", model.isOpen, model.interval);
    }
}];
```

3.2.2.2.3 心率变异性(HRV)检测设置与获取

间隔HRV只可设置60分钟;

方法说明:

```
+(void)setHrvMode:(DHHrvModeSetModel *)model block:(void(^)(int code, id data))block

+(void)getHrvMode:(void(^)(int code, id data))block
```

参数说明:

参数	类型	说明	值
model	DHHrvModeSetModel	类	isOpen: true开/false关 interval: 间隔时间固定60分钟 startHour: 0 固定0不能修改 startMin: 0 固定0不能修改 endHour: 23 固定23不能修改 endMin: 59 固定59不能修改;

调用示例:

```
// 3. Set HRV Monitor(设置HRV监听)
DHHrvModeSetModel *tModeSetModel = [[DHHrvModeSetModel alloc] init];
tModeSetModel.isOpen = YES;
tModeSetModel.startHour = 00; //开始时间固定
tModeSetModel.startMinute = 00;
tModeSetModel.endHour = 23; //结束时间固定
tModeSetModel.endMinute = 59;
tModeSetModel.interval = 60; //固定不可设置
[DHBLECommand setHrvMode:tModeSetModel block:^(int code, id _Nonnull data) {
    if (code == 0){
        NSLog(@"setHrvMode OK");
    }
}];

//3. 获取HRV监听
[DHBLECommand getHrvMode:^(int code, id _Nonnull data) {
    if (code == 0){
        DHHrvModeSetModel *model = data;
        NSLog(@"getHrvMode OK 开关 %d", model.isOpen);
    }
}];
```

3.2.2.2.4 压力检测设置与获取

间隔压力只可设置60分钟;

方法说明:

```
+(void)setStressMode:(DHStressModeSetModel *)model block:(void(^)(int code, id data))block

+(void)getStressMode:(void(^)(int code, id data))block
```

参数说明:

参数	类型	说明	值
model	DHStressModeSetModel	类	isOpen: true开/false关 interval: 间隔时间固定60分钟 startHour: 0 固定0不能修改 startMin: 0 固定0不能修改 endHour: 23 固定23不能修改 endMin: 59 固定59不能修改;

调用示例:

```
// 4. 设置压力监听
DHStressModeSetModel *tModeSetModel = [[DHStressModeSetModel alloc] init];
tModeSetModel.isOpen = YES;
tModeSetModel.startHour = 00; //开始时间固定
tModeSetModel.startMinute = 00;
tModeSetModel.endHour = 23; //结束时间固定
tModeSetModel.endMinute = 59;
tModeSetModel.interval = 60; //固定不可设置
[DHBLECommand setStressMode:tModeSetModel block:^(int code, id _Nonnull data) {
    if (code == 0){
        NSLog(@"setStressMode OK");
    }
}];

//4. 获取压力监听
[DHBLECommand getStressMode:^(int code, id _Nonnull data) {
    if (code == 0){
        DHStressModeSetModel *model = data;
        NSLog(@"getStressMode OK 开关 %d", model.isOpen);
    }
}];
```

3.2.2.2.5 血糖检测设置与获取

间隔血糖只可设置60分钟;

配置表属性: `isDataTypeBloodSugar`

方法说明:

```
+(void)setBloodSugarMode:(DHBloodSugarModeSetModel *)model block:(void(^)(int code, id data))block
```

```
+(void)getBloodSugarMode:(void(^)(int code, id data))block
```

参数说明:

参数	类型	说明	值
model	DHBloodSugarModeSetModel	类	isOpen: true开/false关 interval: 间隔时间固定60分钟 startHour: 0 固定0不能修改 startMin: 0 固定0不能修改 endHour: 23 固定23不能修改 endMin: 59 固定59不能修改;

调用示例:

```
// 5. 设置血糖监听
DHBloodSugarModeSetModel *tModeSetModel = [[DHBloodSugarModeSetModel alloc]
init];
tModeSetModel.isOpen = YES;
tModeSetModel.startHour = 00; //开始时间固定
tModeSetModel.startMinute = 00;
tModeSetModel.endHour = 23; //结束时间固定
tModeSetModel.endMinute = 59;
tModeSetModel.interval = 60; //固定不可设置
[DHBleCommand setBloodSugarMode:tModeSetModel block:^(int code, id _Nonnull
data) {
    if (code == 0){
        NSLog(@"setBloodSugarMode OK");
    }
}]];

//5. 获取血糖监听
[DHBleCommand getBloodSugarMode:^(int code, id _Nonnull data) {
    if (code == 0){
        DHBloodSugarModeSetModel *model = data;
        NSLog(@"getBloodSugarMode OK 开关 %d", model.isOpen);
    }
}]];
```

3.2.2.3 全天检测-同步健康历史数据

同步健康历史数据, 会根据设备支持情况自动同步对应健康数据.

data为数组,为获取到的对应类型的多天数据, 会依次返回对应类型数据, data里不会有多种类型数据.

```
[DHBleCommand startDataSyncing:^(int code, id data){
    NSLog(@"同步完成 %d", code);
} datablcok:^(int code, int progress, id _Nonnull data) {
    if (code == 0) {
        if ([data isKindOfClass:[NSArray class]]) {
            NSArray *array = data;
            for (id model in array) {
```



```

        if ([model isKindOfClass:[DHDailyStepModel class]]) {
            NSLog(@"同步有 计步数据");
        }
        else if ([model isKindOfClass:[DHDailySleepModel
class]]) { //睡眠数据
            NSLog(@"同步有 睡眠数据");
        }
        else if ([model isKindOfClass:[DHDailyHrModel
class]]) { //心率数据
            NSLog(@"同步有 心率数据");
        }
        else if ([model isKindOfClass:[DHDailyBoModel
class]]) { //血氧数据
            NSLog(@"同步有 血氧数据");
        }
        else if ([model isKindOfClass:[DHDailyHrvModel
class]]) { ///HRV数据
            NSLog(@"同步有 HRV数据");
        }
        else if ([model isKindOfClass:[DHDailyPressureModel
class]]) { ///压力数据
            NSLog(@"同步有 Stress数据");
        }
        else if ([model isKindOfClass:[DHDailyBloodSugarModel
class]]) { ///血糖数据
            NSLog(@"同步有 血糖数据");
        }
        else if ([model isKindOfClass:
[DHDailyMuslimCountModel class]]) { ///赞念数据
            NSLog(@"同步有 赞念数据");
        }
    }
}
}
}];

```

3.2.2.4 全天检测-健康数据说明

1. 计步数据 DHDailyStepModel

```

@interface DHDailyStepModel : NSObject

/// 日期时间戳 (秒)
@property (nonatomic, copy) NSString *timestamp;
/// 日期yyyyMMdd
@property (nonatomic, copy) NSString *date;

/// 里程 (米)
@property (nonatomic, assign) NSInteger distance; //当天的总距离
/// 消耗 (卡路里)
@property (nonatomic, assign) NSInteger calorie; //当天总卡路里
/// 步数 (步)

```

```

@property (nonatomic, assign) NSInteger step; //当天总步数

/// 计步项 例:
@[@"index":@0,@"step":@100,@"calorie":@10000,@"distance":@50},...]
/// index (序号) step (步数) calorie (消耗) distance (里程) 单位同上
/// index从0开始, 固定24个item, 对应一天24小时
@property (nonatomic,strong) NSArray <NSDictionary *>*items; //一天时每小时的数据,总24条(今天未满足24条)

@end

```

2. 睡眠数据 DHDailySleepModel

```

@interface DHDailySleepModel : NSObject

/// 时间戳 (秒)
@property (nonatomic, copy) NSString *timestamp;
/// 日期yyyyMMdd
@property (nonatomic, copy) NSString *date;

/// 总时长 (分钟)
@property (nonatomic, assign) NSInteger duration;
/// 入睡时间 (时间戳 (秒))
@property (nonatomic, copy) NSString *beginTime;
/// 醒来时间 (时间戳 (秒))
@property (nonatomic, copy) NSString *endTime;

/// 睡眠项 例: @[@"status":@0,@"value":@60},...]
/// status (睡眠类型) value (时长 (分钟))
/// status (0.清醒 1.浅睡 2.深睡, 3. REM)
@property (nonatomic,strong) NSMutableArray <NSDictionary *>*items;

@end

```

3. 心率数据 DHDailyHrModel;

```

@interface DHDailyHrModel : NSObject

/// 时间戳 (秒)
@property (nonatomic, copy) NSString *timestamp;
/// 日期yyyyMMdd
@property (nonatomic, copy) NSString *date;

/// 心率项 例: @[@"timestamp":@0,@"value":@80},...]
/// timestamp (时间戳 (秒)) value (心率值)
@property (nonatomic,strong) NSMutableArray <NSDictionary *>*items;

@end

```

HRV `DHDailyHrvModel`, 压力 `DHDailyPressureModel`, 血氧 `DHDailyBoModel`, 血糖 `DHDailyBloodSugarModel` 与心率类似可参考对应类

4. Muslim赞念数据 `DHDailyMuslimCountModel`

```
@interface DHDailyMuslimCountModel : NSObject
/// 时间戳 (秒)
@property (nonatomic, copy) NSString *timestamp;
/// 日期yyyyMMdd
@property (nonatomic, copy) NSString *date;

@property (nonatomic, assign) NSInteger muslimcount;

/// 赞念项 例: @[ @{@"timestamp":@0,@"value":@80},...]
/// timestamp (时间戳 (秒)) value (每小时赞念值)
@property (nonatomic, strong) NSMutableArray <NSDictionary *>*items; //注意,
这里的每小时为累加值赞念

@end
```

3.2.3 OTA升级

④ Note

ota升级文件需从厂家生成取得, 确定无误后再进行测试. 防止升级出错变砖.

```
[DHBleCommand startPXIUFileSyncing:[NSData dataWithContentsOfFile:tFilePath]
bleKey:BLE_KEY_ALL_BIN_DATA_PXI block:^(int code, CGFloat progress, id _Nonnull
data) {
    NSLog(@"code %d progress %f data %@", code, progress, data);
}];
```

3.2.4 多运动Workout

④ Caution

支持多运动配置表属性为 `isSupportWorkout3`; 开启多运动后, 设备会进入运动中, APP断开与关闭都不会停止, 只有通过APP或设备主动停止, 所以带多运动功能的设备, 连接后先查询下状态, 确定是否在运动中, 在多运动状态下会影响其它功能使用;

3.2.4.1 获取设备多运动状态

获取设备是否在多运动中; 当前不在运动中时才开启新的运动.

方法说明:

```
+(void)getControlSportWithRing:(void(^)(int code, id data))block
```

参数说明:

WorkoutControlType枚举	类型	说明	值
Workout_Begin	Int	整形	运动开始
Workout_Continue	Int	整形	运动继续
Workout_Pause	Int	整形	运动暂停
Workout_Finish	Int	整形	运动结束

调用示例:

```
[DHBluetoothCommand getControlSportWithRing:^(int code, id _Nonnull data) {
    // @{@"keySportType":@(tSportType), @"keyControlType":@(tControlType)}
    if (code == 0 && [data isKindOfClass:[NSDictionary class]]){
        NSDictionary *tDic = data;
        NSInteger tSportType = [tDic[@"keySportType"] integerValue];
        WorkoutControlType tControlType = [tDic[@"keyControlType"] integerValue];
    }
};
```

3.2.4.2 控制设备进入多运动

控制设备进入多运动, 启动运动.

运动中数据变化通过接收 BluetoothNotificationRingRuningData 通知获取.

方法说明:

```
+(void)controlSportWithRing:(DHSportControlModel *)model block:(void(^)(int code, id data))block
```

参数说明:

参数	类型	说明	值
DHSportControlModel	类		controlType:参考WorkoutControlType sportType: 参考BleActivityMode

运动数据变化通知返回数据说明:

参数	类型	说明	值
ActivityTime	Int	整形	运动持续时间,单位 秒(s);
ActivitySteps	Int	整形	运动中步数
ActivityDistance	Int	整形	运动中产生距离, 单位 米(m);
ActivityCalorie	Int	整形	运动中产生热量, 单位 卡(cal);
ActivityHr	Int	整形	运动中动态心率
ActivityDataType	Int	整形	来源类型, setRingEnterWorkOut也会返回此数据.

调用示例:

```
DHSportControlModel *model = [[DHSportControlModel alloc] init];
model.controlType = Workout_Begin; //开始
model.sportType = tbleActivityMode;
[DHBleCommand controlSportWithRing:model block:^(int code, id _Nonnull data) {
    if (code == 0){
        WorkoutRunningController *runningC = [[WorkoutRunningController alloc]
initWithNibName:@"WorkoutRunningController" bundle:nil];
        runningC.bleActivityMode = tbleActivityMode;
        runningC.controllType = Workout_Begin; //开始
        runningC.modalPresentationStyle = UIModalPresentationOverFullScreen;
        [weakSelf presentViewController:runningC animated:YES completion:^(
        });
    }
}];

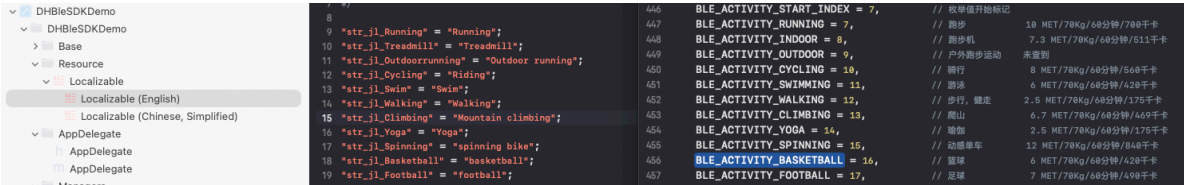
[[NSNotificationCenter defaultCenter] addObserver:self
selector:@selector(ringRuningDataUdpate:)
name:BluetoothNotificationRingRuningData object:nil];

- (void)ringRuningDataUdpate:(NSNotification *)ntf
{
    NSDictionary *tUserInfo = ntf.userInfo;
    if (tUserInfo.count > 0){
        NSInteger duration = [tUserInfo[@"ActivityTime"] integerValue];
        NSInteger totalStep = [tUserInfo[@"ActivitySteps"] integerValue];
        NSInteger activityDistance = [tUserInfo[@"ActivityDistance"]
integerValue];
        NSInteger activityCalorie = [tUserInfo[@"ActivityCalorie"] integerValue];
        NSInteger activityHr = [tUserInfo[@"ActivityHr"] integerValue];
        NSInteger tActivityDataType = [tUserInfo[@"ActivityDataType"]
integerValue];

        NSLog(@"ringRuningDataUdpate tActivityDataType %04X",
(UInt32)tActivityDataType);
        NSInteger hours = duration / 3600;
```

```
NSInteger minutes = (duration % 3600) / 60;
NSInteger seconds = duration % 60;
self.workoutTimeLb.text = [NSString stringWithFormat:@"Time:
%02ld:%02ld:%02ld", (long)hours, (long)minutes, (long)seconds];
self.workoutStepLb.text = [NSString stringWithFormat:@"Steps: %ld",
totalStep];
self.workoutDistanceLb.text = [NSString stringWithFormat:@"Distance: %.2f
Km", floor(activityDistance/1000.0*100)/100];
self.workoutCaloriesLb.text = [NSString stringWithFormat:@"Calories: %.1f
KCal", floor(activityCalorie/1000.0*10)/10];
self.workoutHeartLb.text = [NSString stringWithFormat:@"HeartRate: %ld
bpm", activityHr];
}
}
```

BleActivityMode 对应名字见示例Demo 字符串里定义:



3.2.4.3 控制开启/关闭设备实时通知运动数据

控制开启/关闭设备实时通知运动数据;

运动中数据变化通过接收 `BluetoothNotificationRingRuningData` 通知获取,有时app关闭与进入后台,可告诉设备停止通知数据.

方法说明:

```
+(void)setRingEnterWorkOut:(UInt8)isEnter block:(void(^)(int code, id data))block
```

参数说明:

参数	类型	说明	值
isEnter	Int	整形	1: 开启通知运动数据; 0: 关闭通知运动数据;

调用示例:

```
//退出运动界面
[DHBLECommand setRingEnterWorkOut:0 block:^(int code, id _Nonnull data) {

}];
```

3.2.4.4 获取多运动数据报告

方法说明:

```
+(void)startRingWorkout3Syncing:(void(^)(int code, id data))block dataBlock:
(void(^)(int code, int progress, id data))dataBlock
```

返回数据DHDailySportModel参数说明:

DHDailySportModel类	类型	说明	值
timestamp	NSString		运动开始时间戳
date	NSString		日期yyyyMMdd
viewType	Int		当前运动类型有无步数,步频,配速,距离: 有步频 viewTypeHaveStepFaq: 无步数 viewTypeNoStepNum: 有配速 viewTypeHavePace: 无距离 viewTypeNoDistance:
heartRateItems	数组		当前运动产生的心率列表, 1分钟保存一个;
.....			其它属性见类注释

调用示例:

```
[DHBLECommand startRingWorkout3Syncing:^(int code, id _Nonnull data) {
    NSLog(@"startRingWorkout3Syncing 同步完成 code %d", code);
} dataBlock:^(int code, int progress, id _Nonnull data) {
    if (code == 0) {
        if ([data isKindOfClass:[NSArray class]]){
            NSArray *array = data;
            for (id model in array) {
                if ([model isKindOfClass:[DHDailySportModel class]]) {
                    //保存数据库或其它操作
                }
            }
        }

        NSLog(@"startRingWorkout3Syncing data %zd", array.count);
    }
}
}];
```

SDK修订记录

版本	内容描述	修订日期
1.0.0	1. 添加电子烟新功能;	2024.12.01
1.0.1	2. 添加戒指产品功能;	2025.04.08
1.0.2	3. 添加Muslim定制产品相关功能;	2025.07.23
1.0.3	1. 添加Muslim定制功能; 2. 戒指产品去掉第三方Framework;	2025.08.12
1.0.4	1. 赞念可设置测试值; 2. 启动单次赞念也实时同步数据;	2025.08.21
1.0.5	1. 加设置亮屏时长时关闭睡眠模式功能;	2026.01.05
1.0.6	1. 添加赞念客户新需求;	2026.01.15
V2.0.0_20260118	1. 加sdk 版本号接口; 2. 加功能配置表; 3. 加震动,睡眠模式,消息推送,报警等设置新功能指令; 4. 加多运动功能指令;	2026.01.18
V2.0.0_20260208	1. 加OTA升级加超时处理;	2026.02.08
V2.0.0_20260225	2. add 3.2.1.18 Set Time Format ;	2026.02.25

联系方式 / 技术支持

- 技术支持邮箱 developer@dhhouse88.com